

QUEUE ALGORITHM DESIGN & ANALYSIS WITH JAVA

Customer Service Queue (ระบบคิวลูกค้าศูนย์บริการ)



สมาชิก

นายกษิต อนุอำไพ 68122250028

นางสาวน้ำผึ้ง เอียดวงศ์ 68122250031

นายชลสิทธิ์ จิตรณรงค์ 68122250092

นายณัฐชนันท์ จินดาณิล 68122250102

นายพฤทธราชต์ ดำนิล 68122250104



ส่วนที่1: การวิเคราะห์ปัญหา (PROBLEM ANALYSIS)

ศูนย์บริการมีเคาน์เตอร์ให้บริการทั้งหมด 3 ช่อง (Counter 1, Counter 2, Counter 3) ปัญหาหลักของการจัดคิวคือการบริหารจัดการลูกค้าที่เดินทางเข้ามาใช้บริการประเภทต่างๆ ให้ได้รับการบริการอย่างมีประสิทธิภาพและยุติธรรมที่สุด



1.2 การเลือกและเหตุผลในการใช้ QUEUE

ข้อมูลที่ Queue เก็บ: วัตถุแบบ Customer ซึ่งบรรจุ Attribute สำคัญสำหรับการประมวลผลชนิดของ Queue ที่เหมาะสม: เลือกใช้ FIFO Queue (First-In, First-Out) โดยประยุกต์ใช้ ArrayDeque<Customer> ใน Java เหตุผลที่ต้องใช้ Queue: โครงสร้างข้อมูลแบบ Queue ช่วยการันตีลำดับก่อน-หลังของลูกค้า ป้องกันการแทรกคิว และให้ประสิทธิภาพทางเวลาในการใส่ข้อมูล (Enqueue) และดึงข้อมูล (Deque) ที่คงที่คือ $O(1)$

1.1 องค์ประกอบของระบบ (SYSTEM ELEMENTS)

INPUT: ข้อมูลลูกค้า ประกอบด้วย Customer ID, Arrival Time (เวลาที่มาถึง), Service Type (ประเภทบริการ: GENERAL, FINANCIAL, VIP), และ Service Duration (ระยะเวลาในการให้บริการ) รวมถึงคำสั่งดำเนินงาน (ARRIVE, ASSIGN_COUNTER, SERVICE_COMPLETE, CANCEL, DISPLAY_QUEUE)

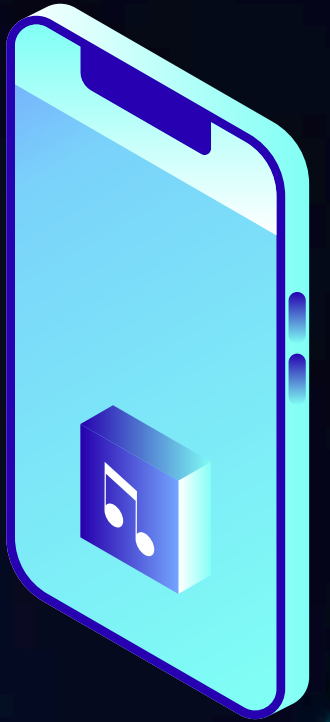
OUTPUT: สถานะการจัดคิว, การมอบหมายลูกค้าเข้าเคาน์เตอร์, สถิติระยะเวลารอคอย (Waiting Time), ความยาวคิว (Queue Length), และการทำงานของเคาน์เตอร์ (Counter Utilization)

Constraints: มีเคาน์เตอร์ให้บริการเพียง 3 ช่อง, ลูกค้าที่ถูกเรียกเข้าเคาน์เตอร์แล้วจะไม่สามารถยกเลิกคิวได้, และการจัดการคิวต้องคำนึงถึงความยุติธรรมตามหลัก First-Come, First-Served (FCFS)



ส่วนที่ 2: การออกแบบอัลกอริทึม (ALGORITHM DESIGN)

2.1 ALGORITHM A: SEPARATE QUEUE (แยกคิวตามเคาน์เตอร์) ลูกค้าที่เดินทางมาถึงจะเลือกเข้าต่อท้ายคิวของเคาน์เตอร์ที่มีจำนวนคนรออยู่ในขณะนั้นน้อยที่สุด



```
// 1. ARRIVE Operation
FUNCTION arrive(customer):
    shortestIndex = 0
    minSize = queues[0].size()

    // หา Queue ที่มีจำนวนคนรอน้อยที่สุด
    FOR i FROM 1 TO queues.length - 1 DO:
        IF queues[i].size() < minSize THEN:
            minSize = queues[i].size()
            shortestIndex = i
        END IF
    END FOR

    queues[shortestIndex].addLast(customer) // เพิ่มลงท้ายคิวที่สั้นที่สุด
END FUNCTION
```

```
// 2. ASSIGN_COUNTER Operation
FUNCTION assignCounter(currentTime):
    FOR i FROM 0 TO counters.length - 1 DO:
        // ถ้าเคาน์เตอร์ว่าง และ มีลูกค้าต่อคิวขงนั้นอยู่
        IF NOT counters[i].isBusy() AND NOT queues[i].isEmpty() THEN:
            customer = queues[i].pollFirst() // ดึงคนหน้าสุดของคิวนั้น
            counters[i].assignCustomer(customer, currentTime)
        END IF
    END FOR
END FUNCTION
```

```
// 3. SERVICE_COMPLETE Operation
FUNCTION serviceComplete(counterId, currentTime):
    counterIndex = counterId - 1
    IF counterIndex >= 0 AND counterIndex < counters.length THEN:
        counters[counterIndex].completeService(currentTime)
    END IF
END FUNCTION
```

```
// 4. CANCEL Operation
FUNCTION cancelCustomer(targetCustomerId):
    FOR EACH queue IN queues DO:
        FOR EACH customer IN queue DO:
            IF customer.id == targetCustomerId THEN:
                queue.remove(customer) // ลบคนออกจากคิวกกลางแถว
                RETURN TRUE
            END IF
        END FOR
    END FOR
    RETURN FALSE
END FUNCTION
```

2.2 ALGORITHM B: SINGLE SHARED QUEUE (คิวกลางเดียว)

ลูกค้าทุกคนต่อคิวกลางคิวเดียวกัน เมื่อเคาน์เตอร์ใดว่างลง จะถึงลูกค้าคนแรกสุดในคิวกลางไปรับบริการทันที

```
// 1. ARRIVE Operation
FUNCTION arrive(customer):
    sharedQueue.offer(customer) // เพิ่มลงท้ายคิวกลางเดียว (Enqueue)
END FUNCTION

// 2. ASSIGN_COUNTER Operation
FUNCTION assignCounter(currentTime):
    FOR EACH counter IN counters DO:
        // ถ้าเคาน์เตอร์ว่าง และ คิวกลางมีคนรออยู่
        IF NOT counter.isBusy() AND NOT sharedQueue.isEmpty() THEN:
            customer = sharedQueue.poll() // ดึงคนหน้าสุดของคิวกลางออก (Dequeue)
            counter.assignCustomer(customer, currentTime)
        END IF
    END FOR
END FUNCTION
```

```
// 3. SERVICE_COMPLETE Operation
FUNCTION serviceComplete(counterId, currentTime):
    counterIndex = counterId - 1
    IF counterIndex >= 0 AND counterIndex < counters.length THEN:
        counters[counterIndex].completeService(currentTime)
    END IF
END FUNCTION

// 4. CANCEL Operation
FUNCTION cancelCustomer(targetCustomerId):
    FOR EACH customer IN sharedQueue DO:
        IF customer.id == targetCustomerId THEN:
            sharedQueue.remove(customer) // ลบคนออกจากคิวกลาง
            RETURN TRUE
        END IF
    END FOR
    RETURN FALSE
END FUNCTION
```


ส่วนที่ 3: QUEUE TRACE (STEP-BY-STEP 10 OPERATIONS)

การจำลองการทำงานอย่างเป็นขั้นตอนจำนวน 10 OPERATIONS บน ALGORITHM B (SINGLE SHARED QUEUE)

Step	Operation	Queue State (Front → Rear)	Counter States (C1, C2, C3)	คำอธิบายเหตุการณ์การเปลี่ยนสถานะ
1	ARRIVE C1 (GENERAL, 5m)	[C1]	Idle, Idle, Idle	C1 เข้าสู่คิวกลางว่างเปล่า อยู่ที่ตำแหน่ง Front
2	ARRIVE C2 (VIP, 3m)	[C1, C2]	Idle, Idle, Idle	C2 ต่อท้าย C1 ตามหลัก FIFO
3	ARRIVE C3 (FINANCIAL, 10m)	[C1, C2, C3]	Idle, Idle, Idle	C3 ต่อท้าย C2 ในคิวกลาง
4	ASSIGN_COUNTER (Time=3)	[]	C1 (Busy), C2 (Busy), C3 (Busy)	เคาน์เตอร์ว่างทั้ง 3 ช่อง ดึง C1, C2, C3 ออกจากคิวกลางเข้าเคาน์เตอร์ตามลำดับ คิวกลางกลายเป็นคิวว่าง
5	ARRIVE C4 (GENERAL, 4m)	[C4]	C1 (Busy), C2 (Busy), C3 (Busy)	เคาน์เตอร์เต็มทุกช่อง C4 จึงต้องรอในคิวกลาง
6	ARRIVE C5 (VIP, 2m)	[C4, C5]	C1 (Busy), C2 (Busy), C3 (Busy)	C5 เข้าต่อท้าย C4 ในคิวกลาง
7	SERVICE_COMPLETE C1 (Time=6)	[C4, C5]	C1 (Idle), C2 (Busy), C3 (Busy)	C1 บริการเสร็จสิ้นที่เคาน์เตอร์ 1 สถานะเคาน์เตอร์ 1 เปลี่ยนเป็น Idle
8	ASSIGN_COUNTER (Time=6)	[C5]	C1 (Busy: C4), C2 (Busy), C3 (Busy)	เคาน์เตอร์ 1 ว่าง ดึง C4 จากหัวคิวเข้าบริการ คิวกลางเหลือเพียง [C5]
9	CANCEL C5	[]	C1 (Busy: C4), C2 (Busy), C3 (Busy)	ค้นหาและยกเลิก C5 ออกจากคิวกลาง คิวกลางกลับมาว่างเปล่า
10	DISPLAY_QUEUE	[]	C1 (Busy: C4), C2 (Busy), C3 (Busy)	แสดงผลคิวกลางปัจจุบัน (Empty Queue) และสถานะเคาน์เตอร์

ส่วนที่ 4: ความถูกต้องของอัลกอริทึม [CORRECTNESS & INVARIANT] FIFO QUEUE INVARIANT

"ก่อนเริ่มและหลังจบทุกๆ Operation สมาชิกที่อยู่บริเวณ Front ของคิวกลาง คือสมาชิกที่เดินทางเข้ามาในระบบก่อนสมาชิกคนอื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล"

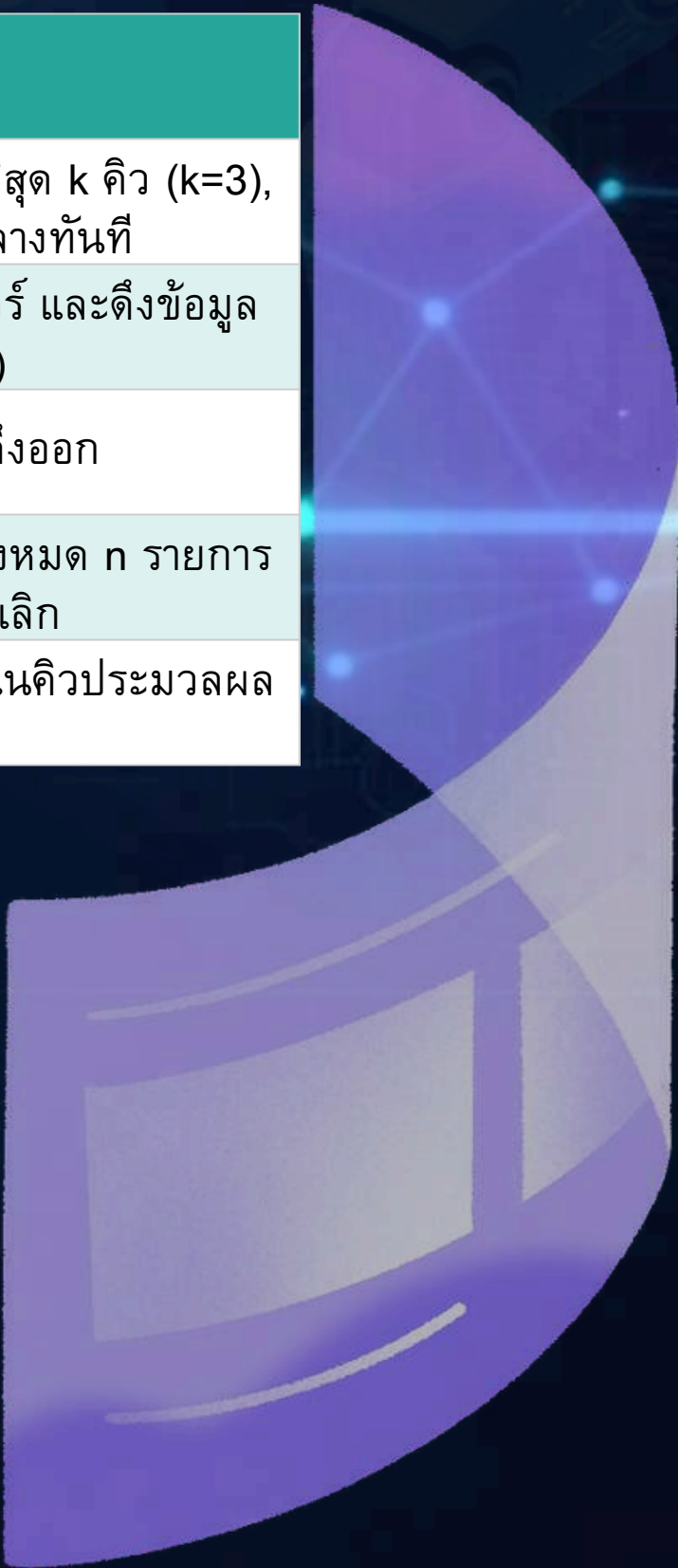
Initialization: เริ่มต้นคิวเป็นคิวว่าง (Empty) เชื่อนไข Invariant เป็นจริงอย่างว่างเปล่า (Vacuously True)

Maintenance: การ arrive() เพิ่มข้อมูลที่ Rear โดยไม่กระทบตำแหน่ง Front การ assignCounter() ตั้งเฉพาะตำแหน่ง Front ออกไปประมวลผลเสมอ ลำดับก่อน-หลังของสมาชิกที่เหลือจึงคงเดิม
Termination: เมื่อจบการทำงาน ลำดับการให้บริการตรงตามลำดับเวลาที่มาถึงอย่างถูกต้องครบถ้วน



ส่วนที่ 5: การวิเคราะห์ TIME COMPLEXITY

Operation	Algorithm A (Separate Queue)	Algorithm B (Single Shared Queue)	คำอธิบาย
enqueue() / arrive()	$O(k) \approx O(1)$	$O(1)$	Algorithm A ต้องวนลูปหาคิวที่สั้นที่สุด k คิว ($k=3$), Algorithm B เพิ่มท้ายคิวกลางทันที
dequeue() / assignCounter()	$O(k) \approx O(1)$	$O(k) \approx O(1)$	ตรวจสอบความว่างของ k เคาน์เตอร์ และดึงข้อมูลหัวคิวออกด้วย $O(1)$
peek()	$O(1)$	$O(1)$	ดูข้อมูลหัวคิวโดยไม่ต้องดึงออก
search() / cancel()	$O(n)$	$O(n)$	ต้องวนลูปค้นหาข้อมูลจากสมาชิกทั้งหมด n รายการในคิวเพื่อทำการยกเลิก
display()	$O(n)$	$O(n)$	การอ่านและพิมพ์ผลสมาชิกทั้งหมดในคิวประมวลผลเป็น $O(n)$





ส่วนที่ 6: การวิเคราะห์ SPACE COMPLEXITY

เมื่อมีจำนวนลูกค้าในระบบสูงสุด n รายการ:

Algorithm A: ใช้ Space Complexity เป็น $O(n + k)$ โดย n คือจำนวนออเบเจต์ Customer รวมทุกคิว และ k คือจำนวนเคาน์เตอร์/คิว ($k=3$) ซึ่งทอนเป็น $O(n)$

Algorithm B: ใช้ Space Complexity เป็น $O(n)$ เนื่องจากใช้อาร์เรย์/โครงสร้างคิวกลางเดียวขนาดแปรผันตาม n และเคาน์เตอร์คงที่ 3 ช่อง



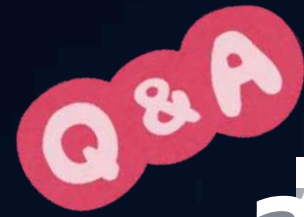
ส่วนที่ 7: ผลการทดลองและการเปรียบเทียบ (EXPERIMENTAL COMPARISON)

การวัดเวลาการประมวลผล (Execution Time) ด้วยวิธี System.nanoTime() ร่วมกับการ Warm-up JVM, การกำหนด Constant Seed, และหาค่าเฉลี่ยจากการทดลอง 5 รอบ ได้ผลลัพธ์ดังนี้

ขนาดข้อมูล (N)	Algorithm A (Separate Queue) [ms]	Algorithm B (Single Queue) [ms]	การวิเคราะห์และอภิปรายผล
100	0.1245 ms	0.0812 ms	การประมวลผลรวดเร็วทั้งคู่เนื่องจาก N มีขนาดเล็ก
1,000	0.8920 ms	0.5410 ms	Algorithm B เร็วกว่าเล็กน้อย เนื่องจากไม่ต้องคำนวณหาคิวที่สั้นที่สุด
10,000	7.4150 ms	4.2100 ms	เวลาเติบโตเป็นเส้นตรงตามทฤษฎี $O(N)$ Algorithm B รักษาระดับ Overhead ได้ต่ำกว่า
50,000	32.8500 ms	19.6200 ms	ผลลัพธ์สอดคล้องกับทฤษฎี Big-O การจัดการคิวกลางเดียวมี Overhead ต่ำกว่าอย่างชัดเจน

ส่วนที่ 8: ตารางเปรียบเทียบเชิงสรุป (ALGORITHM COMPARISON)

หัวข้อการเปรียบเทียบ	Algorithm A: Separate Queue	Algorithm B: Single Shared Queue
วิธีการทำงาน	แยกคิวตามเคาน์เตอร์ ลูกค้าเลือกต่อคิวที่สั้นที่สุด	คิวกลางเดียว ทุกคนต่อแถวเดียว เคาน์เตอร์ว่างจึงเรียก
Time Complexity (Arrive)	$O(k)$	$O(1)$
Space Complexity	$O(n + k)$	$O(n)$
เวลารอคอยเฉลี่ย (Waiting Time)	สูงกว่า (หากมีเคสค่อขวดในแถวใดแถวหนึ่ง)	ต่ำกว่าและสม่ำเสมอกว่าอย่างเห็นได้ชัด
ความยุติธรรม (Fairness)	ปานกลาง (อาจเกิดการแซงคิวข้ามแถวโดยไม่ได้ตั้งใจ)	สูงสุด (การันตี FIFO ตามลำดับเวลาเดินทางมาถึง 100%)



ส่วนที่ 9: สรุปคำถามท้ายเคสปฏิบัติการ



คำถาม

เหตุใดธนาคารและสนามบินจำนวนมากจึงนิยมใช้ Single Queue + Multiple Servers?

คำตอบ

รับประกันความยุติธรรม (Strict Fairness): การใช้ Single Queue ช่วยรักษากฎ First-Come, First-Served (FCFS) ได้ 100% ผู้ที่มาก่อนจะได้รับบริการก่อนเสมอ ขจัดปัญหาผู้ใช้บริการรู้สึกไม่พอใจเมื่อแถวข้างๆ เคลื่อนตัวเร็วกว่า

ลดเวลารอคอยเฉลี่ยและป้องกันคอขวด (Queueing Theory Optimization): ตามทฤษฎีการจัดคิว (M/M/c Model) คิวกลางเดียวจะช่วยขจัดปัญหาเคาน์เตอร์หนึ่งว่าง ขณะที่อีกเคาน์เตอร์ติดเคสที่ใช้เวลานาน (Bottleneck) ส่งผลให้ค่า Counter Utilization สูงขึ้น และเวลารอคอยเฉลี่ยรวมของระบบลดลง

ลดความเครียดทางจิตวิทยาของผู้ใช้บริการ (Reduced Psychological Anxiety): ผู้ใช้บริการไม่ต้องแบกรับความเสี่ยงในการ "เลือกแถวผิด" และไม่ต้องพยายามสลับแถวไปมา (Jockeying) ทำให้ประสบการณ์การรับบริการดีขึ้นอย่างมีนัยสำคัญ

ส่วนที่ 10: เปรียบเทียบ ALGORITHMS

ตารางเปรียบเทียบประสิทธิภาพ โครงสร้างข้อมูล และคุณลักษณะการทำงานระหว่าง
Algorithm A (Separate Queues) และ Algorithm B (Single Shared Queue)

ประเด็น	Algorithm A (Separate Queues)	Algorithm B (Single Shared Queue)
Data Structure	List<Deque<Customer>> (โครงสร้างแบบหลายคิว แยกตามแต่ละเคาน์เตอร์)	Queue<Customer> หรือ ArrayDeque<Customer> (โครงสร้างแบบคิวกลางเดียว)
หลักการ	แยกคิวตามเคาน์เตอร์ ลูกค้าจะเลือกเข้าต่อคิวช่องที่มีจำนวนคนรอน้อยที่สุด ณ เวลาที่	ใช้คิวกลางรวมเพียงคิวเดียว ลูกค้าทุกคนต่อคิวเดียวกัน เมื่อมีเคาน์เตอร์ใดว่างจะเรียกคิวแรก
Enqueue	ค้นหาคิวที่สั้นที่สุด $O(k)$ แล้วนำข้อมูลเข้าต่อท้ายคิวนั้น $O(1)$	นำข้อมูลเข้าต่อท้ายคิวกลางเดียวได้ทันที $O(1)$
Dequeue	ดึงข้อมูลออกจากหัวคิวของเคาน์เตอร์เฉพาะที่ว่าง $O(1)$	ดึงข้อมูลออกจากหัวคิวกลางเดียวเพื่อส่งมอบให้เคาน์เตอร์ที่ว่าง $O(1)$
Time Complexity	Arrive: $O(k)$ Assign: $O(k)$	Arrive: $O(1)$ Assign: $O(k)$
Space Complexity	$O(n + k)$ หรือทอนทวิคูณเป็น $O(n)$	$O(n)$
Fairness	ปานกลาง: อาจเกิดกรณีคิวข้างๆ ได้รับบริการเร็วกว่าแม้ว่าจะเดินทางมาทีหลัง	สูงสุด (100%): การันตีหลักการ First-Come, First-Served (FCFS) ใครมาก่อนได้รับบริการ
ข้อดี	การจัดการทางกายภาพหางานง่าย ไม่จำเป็นต้องมีจอประกาศคิวกลาง	ขจัดปัญหาเคาน์เตอร์ว่างงานขณะที่มีผู้รอรับบริการ (Max Utilization) ลดระยะเวลารอคอยเฉลี่ยรวมของระบบลง
ข้อจำกัด	เกิดคอขวด (Bottleneck) ได้ง่ายหากคิวบางช่องเจอเคสที่ใช้เวลานาน	ต้องมีระบบจอแสดงผลหรือระบบเรียกคิวอัตโนมัติเข้ามาช่วยจัดการ ต้องจัดเตรียมพื้นที่สำหรับเขตรอคอยกลาง (Waiting Area)
เหมาะกับกรณี	ร้านค้าขนาดเล็ก, ซูเปอร์มาร์เก็ตที่มีช่องแคชเชียร์แยกชัดเจน, จุดบริการที่ขั้นตอน	ธนาคาร, สนามบิน, ศูนย์บริการลูกค้า (Service Center), โรงพยาบาล

•จุดการนำเสนอ•

